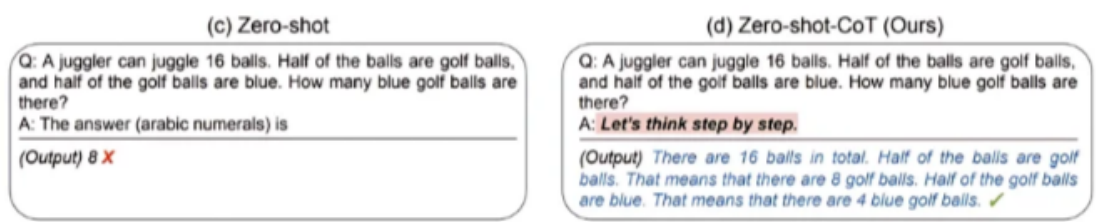


14种常见的Prompt策略

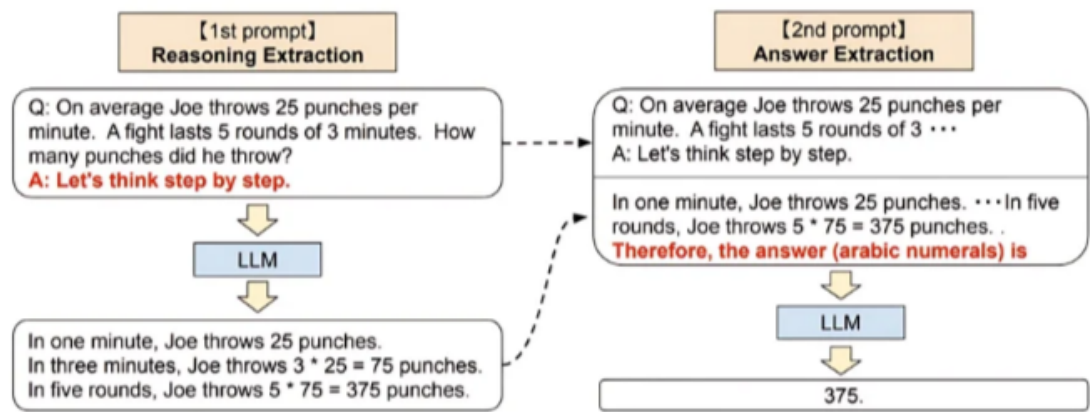
在大型语言模型（Large Language Models, LLMs）的应用中，Prompt 策略是一种重要的技术，用于指导模型生成期望的输出。以下列举了14种常见的Prompt策略：

1. Zero-Shot Prompting

零样本提示是指模型在不看到任何特定任务的示例的情况下，直接根据自然语言指令完成任务的策略。这种策略依赖于模型对指令的理解能力和泛化能力。例如，可以直接询问模型：“解释量子物理学的基本原理。”，而不提供任何相关的示例。



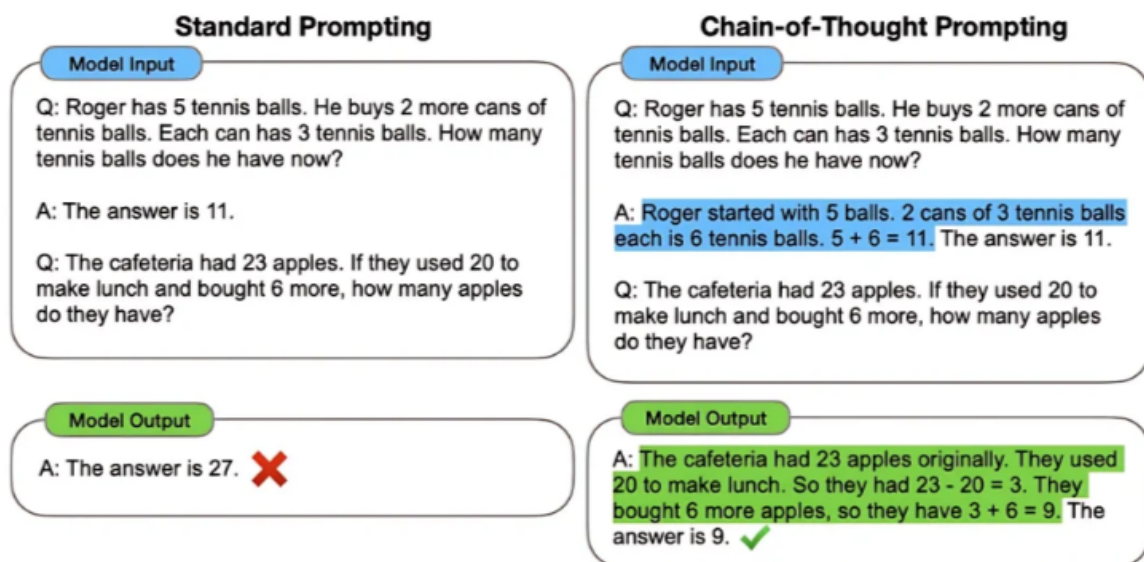
在问题最后加入 Let 's think step by step 后，生成了正确答案



左侧气泡表示基于提示输出的第一次的结果，右侧气泡表示其收到了第一次结果后，将最开始的提示一起拿去运算，最后得出了正确的答案

2. Few-Shot Prompting

少样本提示提供几个与任务相关的示例，然后要求模型完成类似的新任务。这种策略利用了模型的学习和泛化能力，通过少量的示例来指导模型的行为。例如，给出几个产品评论和相应的情感标签，然后让模型对新的评论进行情感分析。



加入解释后，输出的结果就正确了

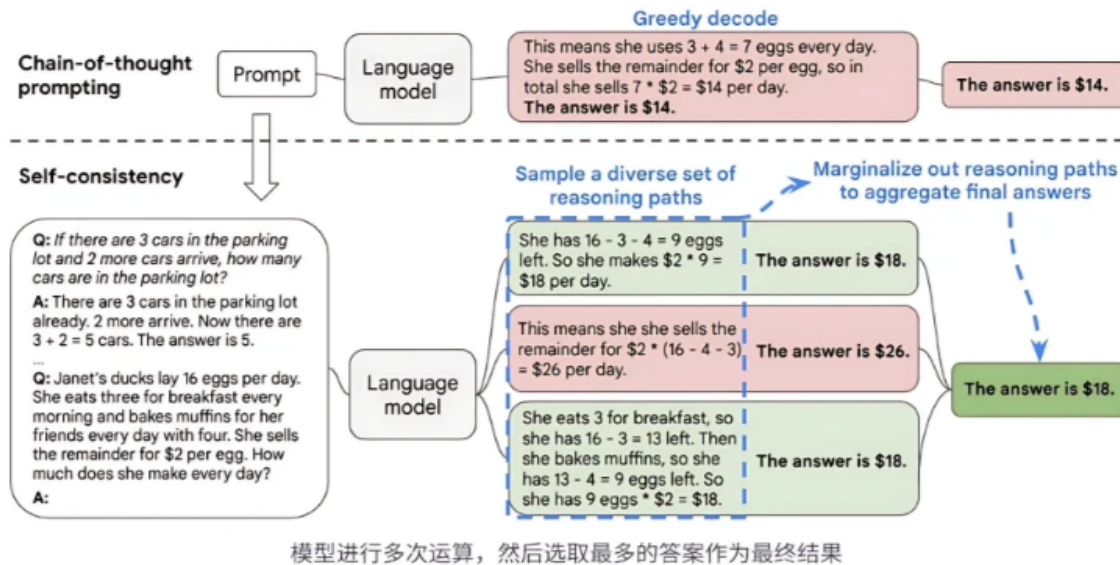
3. Chain-Of-Thought Prompting

思维链提示要求模型在给出答案之前，先展示其解题的中间步骤和推理过程。这种方法有助于提高模型的可解释性和准确性，尤其是在解决需要复杂推理的问题时。

```
1 from transformers import pipeline
2
3 # 初始化连续对话系统
4 def initialize_cot_model(model_name):
5     cot_model = pipeline('text-generation', model=model_name)
6     return cot_model
7
8 # 进行连续对话，模仿CoT
9 def cot_dialogue(cot_model, prompt):
10     response = cot_model(prompt, max_length=50)[0]['text']
11     return response
12
13 # 使用指定的模型名称初始化模型
14 model_name = 'gpt-3' # 例如，可以使用gpt-3或其他任何支持文本生成的模型
15 cot_model = initialize_cot_model(model_name)
16
17 # 提供提示词并开始对话
18 prompt = "你是一个人工智能程序。请像人类一样思考并回答以下问题："
19 question = "1 + 1 等于多少？"
20 full_prompt = prompt + question
21 response = cot_dialogue(cot_model, full_prompt)
22
23 print(f"Question: {question}")
24 print(f"Answer: {response}")
```

4. Self-Consistency

自一致性策略通过多次执行推理过程并选择最一致或最频繁出现的答案来提高模型的输出质量。这种方法可以减少模型在不同推理路径上产生的分歧。



5. Generated Knowledge Prompting

生成知识提示首先让模型生成与任务相关的背景知识或信息，然后使用这些信息来解决问题。这种方法对于需要特定领域知识的任务特别有用，如医学诊断或法律咨询。

```

1 # 定义一个简单的问答系统
2 QA_PAIRS = {
3     "你好": "你好!",
4     "今天的天气如何": "不好说, 因为我是一个人工智能.",
5     "编程语言排行": "Python 是最流行的编程语言."
6 }
7
8 def generate_answer_from_prompt(prompt):
9     for question, answer in QA_PAIRS.items():
10         if prompt == question:
11             return answer
12     return "抱歉, 我不知道这个问题的答案."
13
14 # 使用示例
15 prompt = "编程语言排行"
16 answer = generate_answer_from_prompt(prompt)
17 print(answer)

```

6. Automatic Reasoning & Tool Use (ART)

自动推理和工具使用策略鼓励模型在解决问题时自动使用逻辑推理和外部工具。例如，模型可能会使用数据库查询或网络搜索来获取解决问题所需的信息。

7. Self-Ask Prompting

自我提问提示训练模型在解决问题时提出自己的问题。这可以帮助模型更深入地理解问题，并在必要时分解复杂问题。

```

1 class LLM:
2     def __init__(self):
3         pass
4
5     def generate(self, prompt):
6         # 假设generate方法用于生成文本
7         return "这是对问题 '{}' 的回答".format(prompt.split(':')[0].strip())
8 # 创建一个模型实例
9 model = LLM()
10 # 用户提出问题
11 question = "这是一个示例问题"
12 # 获取模型的回答
13 answer = model.generate(prompt)
14
15 # 打印结果
16 print(answer)

```

8. Sequential Prompting

顺序提示通过一系列的提示逐步引导模型解决问题。每个提示都是基于前一个提示的输出，这种方法适用于需要多步骤推理的任务。

```

1 from transformers import AutoModel, AutoTokenizer, AutoModelForCausalLM
2
3 # 加载预训练的模型和分词器
4 tokenizer = AutoTokenizer.from_pretrained("t5-base")
5 model = AutoModelForCausalLM.from_pretrained("t5-base")
6
7 def generate_sequence(input_str, model=model, tokenizer=tokenizer,
8                       max_length=200, temperature=0.7):
9     inputs = tokenizer.encode(input_str, return_tensors='pt')
10    outputs = model.generate(inputs, max_length=max_length,
11                             temperature_threshold=temperature)
12    return tokenizer.decode(outputs[0], skip_special_tokens=True)
13
14 # 用户连续提问或指令
15 user_input = "你好，今天天气怎样？"
16 response = generate_sequence(user_input)
17 print(response)
18
19 # 继续提问或指令
20 user_input += " 能否推荐一个餐馆？"
21 response = generate_sequence(user_input)
22 print(response)

```


9. Symbolic Reasoning

符号推理结合了符号操作和语言模型的能力，使模型能够处理涉及数学、逻辑或编程的任务，这些任务通常需要精确的符号操作。

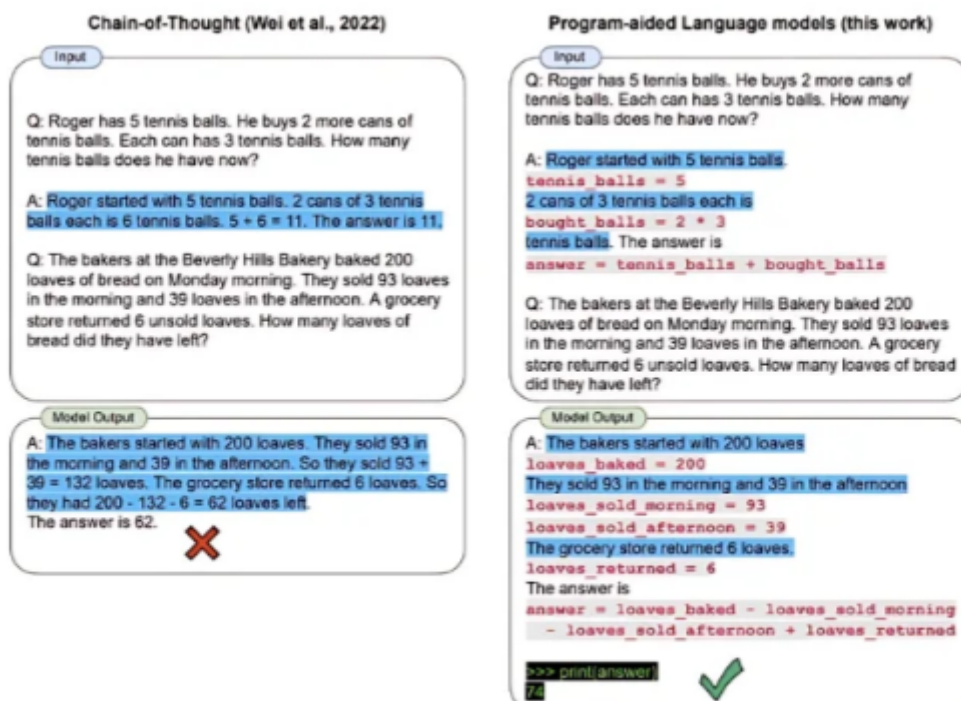
10. Iterative Prompting

迭代提示涉及反复修改和细化提示，以逐步改进模型的输出。这种方法可以用于优化模型的回答，直到达到满意的准确性。

```
1 # 假设我们有一个大型语言模型对象 `language_model`
2 def iterative_prompting(language_model, prompt, max_iterations=5):
3     response = language_model.generate(prompt)
4     for _ in range(max_iterations):
5         prompt = f"{prompt} {response}"
6         response = language_model.generate(prompt)
7     return response
```

11. Program-Aided Language Models

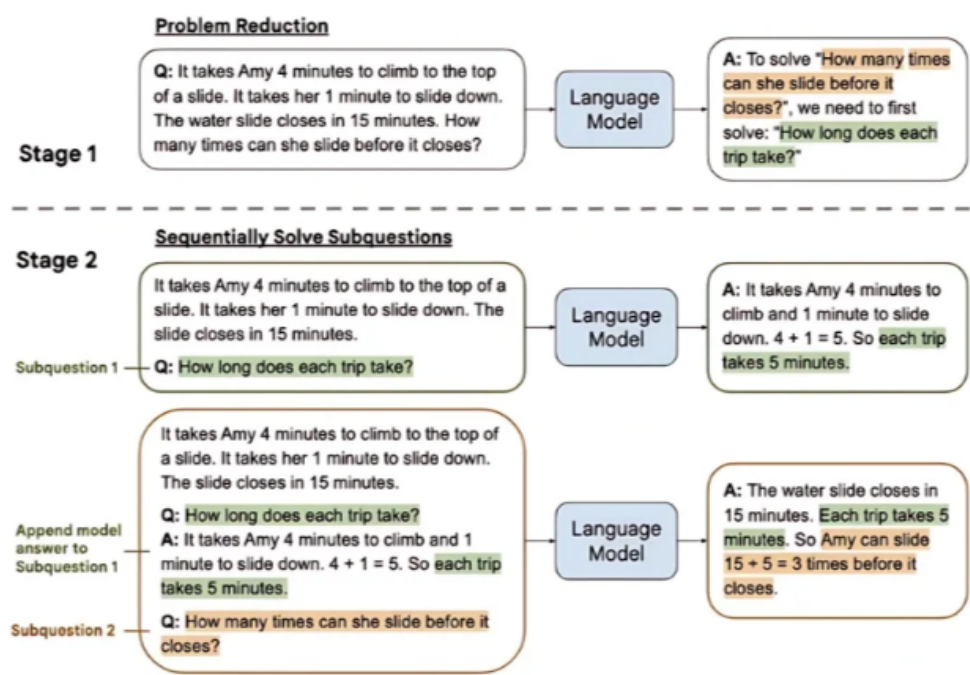
程序辅助语言模型结合了编程逻辑和语言模型，使得模型能够执行需要编程知识的任务，如代码生成或修复。



模型使用代码来运算和思考，最后答案就是正确的

12. Least-To-Most Prompting

从最少到最多提示从最简单的子任务开始，逐步增加任务的复杂性。这种方法有助于模型逐步建立解决问题的能力。



在问题分解阶段中，模型的输入包括 $k \times [\text{原始问题}, \text{子问题列表}]$ 的组合，以及要测试的原始问题；
在逐步解决子问题阶段中，模型输入包括 $k \times [\text{原始问题}, m \times (\text{子问题}, \text{子答案})]$ 元组，要测试的原始问题和当前子问题。

13. Meta-Prompting

元提示是一种策略，其中模型学习如何构建自己的提示来解决问题。这种方法可以使得模型更加灵活和通用。

```

1 from transformers import pipeline
2
3 # 初始化一个使用meta-prompting的问答模型
4 def initialize_model_with_meta_prompt(model_name, meta_prompt):
5     model = pipeline('text-generation', model=model_name)
6     model.prompt = meta_prompt
7     return model
8
9 # 使用meta-prompting进行问答
10 def answer_question_with_meta_prompt(model, question):
11     return model(f'{model.prompt} {question}', max_length=50)[0]['text']
12
13 # 示例使用
14 model_name = 'google/xlnet-base-cased'
15 meta_prompt = "以下是对查询的回应:"
16
17 # 初始化模型
18 model = initialize_model_with_meta_prompt(model_name, meta_prompt)
19
20 # 进行问答
21 question = "地球上的最高山是哪座山?"
22 answer = answer_question_with_meta_prompt(model, question)
23 print(answer)

```

14. ReAct Prompting

反应提示结合了推理、行动和沟通的提示策略。例如，模型在解决问题时可能需要进行外部搜索、执行计算或与用户交互。

```

1 from transformers import LLMController, LLMOutput
2
3 # 初始化LLM控制器
4 controller = LLMController.from_pretrained("microsoft/LLaMA-7B-hf")
5
6 # 定义ReAct Prompting的函数
7 def react_prompt(prompt: str, temperature: float = 0.7):
8     # 使用ReAct Prompting的提示模板
9     prompt = f"ReAct\n> {prompt}\n\n"
10    # 使用LLM控制器生成输出
11    output = controller.generate(prompt, max_length=50,
12                                temperature=temperature)
12    return output
13
14 # 使用ReAct Prompting生成内容
15 prompt = "写一个关于太阳系的科学概述"
16 output = react_prompt(prompt)
17
18 # 打印生成的输出
19 print(output)

```

每种策略都有其特定的应用场景和优势，研究人员和开发者可以根据任务需求选择最合适的提示策略。随着模型和算法的发展，这些策略也在不断地被改进和扩展。